

Setting Up an EDI Vendor in SSCS CDB: A Comprehensive Guide

1. EDI Onboarding and Vendor Creation in SSCS CDB

Vendor Setup: The first step is to create or configure the vendor record in SSCS's Computerized Daily Book (CDB) back-office system with EDI in mind. In the CDB vendor maintenance module, add the new vendor (if not already in your system) and note the **Vendor ID/Code** you assign – this code will be used to identify EDI files for that vendor. Ensure the vendor's details (name, address, account numbers) are entered accurately. You may need to flag the vendor as **EDI-enabled** in the configuration (some SSCS versions have an option or checkbox to indicate the vendor will use EDI for invoices). Also, obtain the vendor's **EDI specifications** (file format, communication method, etc.) during onboarding – SSCS maintains a list of “approved EDI vendors” which have known formats and transport methods ¹ ². If your vendor is in this list, use the known format; if not, coordinate with SSCS support to map the new vendor's format.

Communication and Credentials: Decide how EDI data will be exchanged. SSCS CDB supports EDI invoice ingestion via file drop or email/AS2 delivery. In many cases, SSCS provides an **“edidelivery.com” email address** for your site or company (for example, `v<StoreID>s1@edidelivery.com`) – vendors can send EDI files to this address, and the CDB will retrieve and process them ³ ⁴. If using this method, ensure the vendor's IT team has the correct email and can send test files. For direct file transfer, some vendors set up FTP/SFTP or AS2 connections. SSCS supports AS2 transfers (secure EDI over internet) for many large suppliers – in fact, numerous wholesalers (e.g. Coca-Cola, Core-Mark, Frito-Lay, etc.) use AS2 to send invoices directly to CDB ⁵ ⁶. If using AS2 or FTP, work with the vendor to exchange connection credentials and test connectivity.

Testing and Approval: Before going live, perform a **test EDI exchange**. This usually involves the vendor sending a sample invoice (or you placing a small test order and the vendor sending the resulting EDI invoice). Use CDB's EDI import function (see below) to import the file and verify that it is recognized. SSCS maintains a **list of accepted EDI formats and vendors**, and your file must conform to one of these structures to import correctly ⁷ ⁸. If any issues arise (e.g. fields not aligning, file rejected), it may be necessary to adjust the mapping or involve a **data translation service**. SSCS works with translation partners (like ChimpKey or Deep Water Software) that can convert vendor-specific formats (PDF, custom CSV, etc.) into the required format ⁹ ¹⁰. Once the sample files import without errors, you can proceed to production onboarding. At this point, create any **cross-references** needed in CDB (for example, linking the vendor's internal account number to your vendor record, or setting the vendor's default EDI file name pattern if required).

In summary, vendor onboarding involves creating the vendor in CDB, configuring EDI parameters (ID, format, delivery method), and testing with sample data. SSCS support can provide spec sheets or guidance for new vendors – and if the vendor is one of SSCS's many “approved EDI vendors,” much of the mapping is predefined ¹¹ ¹.

2. Configuring EDI Pricing and Inventory Synchronization

One of the biggest benefits of EDI in SSCS is automatic **price book and inventory updates**. Configuration is needed to ensure that as EDI invoices (and any accompanying item files) are received, the system updates item records (cost, price, etc.) and inventory counts appropriately.

Price Book Integration: Many vendors provide not only invoices but also **item catalogs or price change files**. In SSCS, these are often handled via the *Central Price Book (CPB)* module or the “Outside Updates” mechanism. For each EDI vendor, you can configure a vendor-specific import in CPB that maps to the vendor’s item update format ¹² ¹³. For example, SSCS accepts vendor pricebook files in formats like **PDI “7500” series records or NAXML ItemSync XML** ¹⁴. In the CPB setup, you specify the vendor’s format and the target price book zone (or your local store) to which the updates apply ¹⁵ ¹⁶. When an EDI item file is delivered (usually into the same EDI folder or via email as the invoices), the system will import it into an “Outside Updates” queue. You can review these outside updates (new items, cost changes, etc.) in the CDB interface before applying them. The configuration should also include whether to automatically **push changes to the store’s price book** or wait for manual approval ¹⁷ ¹⁸. Best practice is to review new items or price changes using the CPB or inventory management screens – SSCS can highlight items with cost changes or new items from deliveries ¹⁹.

Inventory Quantity Sync: Each EDI **invoice** will contain the quantities of products delivered (or this can be inferred – see Section 7). When you process an EDI invoice in CDB, the system will add those quantities into your inventory on-hand for each item, just as if you manually received them. To ensure accurate inventory synchronization, confirm that **each EDI item is correctly matched to an item in your CDB database** (usually by UPC or item code). During onboarding, link any vendor-specific item codes to your internal item records (if the UPC is not used as the key). If an EDI invoice contains an item that CDB doesn’t recognize, the system can either create a placeholder new item (if configured to auto-create) or flag it for your attention. You’ll want to configure the “**Create New Items**” setting based on your preference (more on this in Section 7 and 9). For initial setup, many retailers choose to **not auto-create** new items until they’ve done one pass of manual review – instead, they let the invoice import all known items, then separately handle any exceptions for unknown items.

Vendor-Specific Pricing Rules: Some vendors (especially for tobacco, alcohol, or grocery) might include additional pricing info like promotional allowances or suggested retail prices. SSCS’s EDI format and CPB support fields like **retail price (List Price)** and can trigger alerts when a vendor’s price change is detected ¹⁹. During setup, decide how you want to use this: e.g., if the vendor’s **retail price suggestions** should update your shelf prices or just be advisory. The system can alert you when a vendor’s cost change occurs so you can adjust your retail price accordingly ¹⁹. For inventory sync, also verify if the vendor provides **case-pack information** (case size) in the EDI data; if not, ensure your item records have the correct pack size. This allows the system to correctly increment unit inventory. For example, if an invoice says 1 case of soda delivered and your system knows a case is 24 units, CDB will add 24 to inventory.

Overall, configuring pricing and inventory sync means mapping the vendor’s data into your price book structure and deciding on automation level. By setting up the vendor’s **item import format** in CPB and verifying all items, you ensure that EDI deliveries will seamlessly update product info and stock counts without manual data entry ²⁰ ¹⁸. It’s wise to run an initial small delivery to confirm that costs and quantities post correctly to inventory and that any price changes appear in the appropriate reports (for instance, check an “**Outside Updates**” report or **inventory adjustment log** after import).

3. Electronic Invoicing Setup (NAXML and Other Formats)

EDI Invoice Format: SSCS CDB is compatible with multiple EDI invoice formats, but it **prioritizes the Conexus/PCATS NAXML standard** for retail invoices ²¹ ²². NAXML (NACS XML) is an XML schema specifically designed for convenience store data exchange, including invoices, and is widely used in the industry to simplify EDI integration ²¹ ²³. Most approved SSCS vendors use either NAXML or a variant of it for electronic invoices. In practice, a NAXML invoice is an XML file containing the header, line item details, and totals of the invoice (analogous to an X12 810, but in XML form). SSCS's expected structure for invoice XML is often referred to as an **"ItemSynch" or "Item/Price"** format – essentially a business document that lists items delivered with their costs and prices ⁸ ²⁴. (Don't be confused by the name *ItemSynch*; in this context it carries invoice information as well as item data.)

Core Fields in NAXML Invoice: At minimum, the electronic invoice file must include key identifiers like **Vendor ID, Invoice Number, Invoice Date**, and line-item records with **item codes, descriptions, quantities or quantity indicators, unit costs, and extended totals** ²⁵ ²⁶. SSCS's schema also uses an `<InvoiceTotals>` summary section for cross-check (subtotal, tax, total, etc.) ²⁷ ²⁸. For example, a simplified NAXML invoice might have a structure like:

```
<ItemSynch version="2.0" timestamp="2025-08-25T15:12:38Z" vendor="VENDORCODE">
  <VendorInfo>
    <VendorID>VENDORCODE</VendorID>
    <VendorName>Vendor Name Inc.</VendorName>
    <InvoiceNumber>123456</InvoiceNumber>
    <InvoiceDate>2025-08-25</InvoiceDate>
    <TotalItems>10</TotalItems>
    <StoreLocationID>MY_STORE_001</StoreLocationID>
    <TransmissionDate>2025-08-25</TransmissionDate>
  </VendorInfo>
  <Items>
    <Item>
      <PLU>01234567890</PLU>
      <ItemName>Example Product 1</ItemName>
      <!-- Quantity may be implicit (see below) -->
      <Price>59.94</Price>
      <Cost>47.95</Cost>
      <Category>BEVERAGES</Category>
      <Size>6x1L</Size>
      <VendorItemCode>ABC123</VendorItemCode>
      <LastUpdated>2025-08-25T15:12:38Z</LastUpdated>
      <Status>Active</Status>
      <UPC>012345678905</UPC>
      <VerifoneUPC>01234567890</VerifoneUPC>
      <UPCVerified>true</UPCVerified>
    </Item>
    <!-- ...more <Item> entries... -->
  </Items>
```

```

<InvoiceTotals>
  <SubTotal>479.50</SubTotal>
  <Tax>0.00</Tax>
  <Total>479.50</Total>
  <ItemCount>10</ItemCount>
  <RetailTotal>599.40</RetailTotal>
</InvoiceTotals>
<ProcessingInstructions>
  <ImportType>ItemPrice</ImportType>
  <UpdateExisting>true</UpdateExisting>
  <CreateNew>false</CreateNew>
  <NotifyOnCompletion>true</NotifyOnCompletion>
  <UPCCoverage>90.0%</UPCCoverage>
</ProcessingInstructions>
</ItemSynch>

```

This is an example format showing how data is organized. In practice, **SSCS will provide the exact spec or a template for the invoice XML**. The example above aligns with the typical SSCS NAXML invoice template ²⁹ ³⁰. Notably, SSCS expects the file in **UTF-8 XML with a “.xml” extension** and a specific naming convention (often the vendor or invoice number is in the filename). The file can be transmitted as an email attachment or dropped into the system's EDI folder (with a naming pattern the system watches) ⁷ ³¹.

Alternate Formats (X12, CSV): In addition to NAXML XML, SSCS can accept other formats **if they match a known spec**. For legacy or certain vendors, SSCS supports **X12 EDI** transactions – for example, it has documented support for the X12 894 *Product Delivery/Return Base Record* (often used as a Direct Store Delivery pre-delivery notice or ASN) ¹⁰. A few vendors in the approved list use X12-894 via AS2 ¹⁰. While the list doesn't explicitly mention X12 810 (invoices), SSCS can work with EDI providers like TrueCommerce to handle 810 Invoices and other standard documents ³² ³³. In practice, many suppliers send an XML (NAXML) directly, but if a vendor can only do X12, you might use a translation service that converts the 810 into the SSCS XML format behind the scenes. SSCS also historically supported **fixed-width or delimited text** invoice files for some vendors. For instance, some vendors use the PDI “7500 series” text file format for invoices or item updates ¹⁴. Others might use CSV. These are usually handled by reading them into the same EDI folder – the CDB will parse them according to the predefined format for that vendor. The *Central Price Book User's Guide* notes that vendor files can be *.xml or other extensions, and you can even configure a file mask if needed (e.g. to only pick up .xml files)* ³⁴. Example:* A small local vendor could email a CSV of invoice lines; by working with SSCS support, you could map that CSV to the required columns and have it imported just like a standard invoice (often through an intermediary conversion).

NAXML Versions: Be aware of **version compatibility** of the formats. SSCS's NAXML support aligns with the Connexus “EB2B” retail merchandise schema (which has evolved from PCATS). The `ItemSynch version="2.0"` in the XML header is an indicator of the schema version; SSCS will advise if a different version is needed for certain vendors. Generally, stick to the schema provided by SSCS or the vendor's spec sheet. If a vendor is using an older version of NAXML or a slightly different schema, test carefully – in some cases minor tweaks (like tag names or structure) might be required. Always **validate the XML against the expected schema** if possible. SSCS as a Connexus member ensures backward compatibility to earlier NAXML versions for core elements ²², but new fields in newer versions might be ignored if SSCS's importer

isn't updated for them. When in doubt, consult SSCS support to confirm the format. (We'll discuss confirming version compatibility in Section 10.)

Electronic Invoice Flow: Once the format is decided and files are being sent, the workflow is: the vendor transmits the invoice file (via email attachment, AS2 push, or drops it on an FTP which you then place in the EDI folder). Then, in the CDB software, you use the **EDI Import utility** – often labeled “Receive and Convert” – to process incoming EDI files. The system will retrieve the file (download from the email or detect it in the folder) and convert it into an internal format (populating the Accounts Payable and inventory records). For example, after sending a NAXML invoice file to the edidelivery email, you would click “Receive and Convert” in CDB's EDI Conversion screen, and the system would report downloading the message and converting the invoice ³⁵ ³⁶ . The invoices then typically appear in the **Open Invoices list** (or a pending invoice list) in the Accounts Payable module for review/posting. At this stage, the electronic invoice is in the system – including all line items, quantities, and costs – just as if you had entered an invoice by hand, but without manual data entry ³⁷ ³⁸ .

In summary, configure the electronic invoice format to match SSCS's specs (preferably NAXML XML), verify the vendor is sending in that format (or set up translation), and use the CDB's import function to bring those invoices in. This provides a **paperless invoice process** – as SSCS advertises, the vendor's EDI invoice can be transmitted directly to CDB, avoiding the need to key in paper invoices ¹¹ .

4. Process for Handling Purchase Orders and ASNs (Advanced Shipping Notices)

Beyond just receiving invoices, SSCS CDB can also participate in the **ordering side of EDI** – specifically by generating electronic Purchase Orders and processing Advance Shipping Notices (ASNs or “pre-delivery notices”). Here's how those components work:

Computer-Assisted Ordering (Generating EDI POs): SSCS's inventory module features **Computer Assisted Ordering (CAO)**, which allows you to create recommended orders based on sales history or min/max stock levels ³⁹ ⁴⁰ . Once you review or edit the suggested reorder, the system can generate a purchase order file to send to the vendor. For EDI-enabled vendors, this PO can be output in a specific format and transmitted electronically. The format for EDI orders can vary by vendor – some use an XML order schema (part of the NAXML standards, analogous to an X12 850), while others accept a simple text file or even an email with items. SSCS maintains a list of “**Approved CAO Vendors**” which details how orders are sent for each (e.g. via FTP, email, etc.) ² ⁴¹ . For example, **AMCON, Eby-Brown, Farner-Bocken** and many others use an FTP drop for orders, whereas some (like J. Polep) use email or web portal for order transmission ² ⁴¹ . During setup of a CAO vendor, you will configure the **order transport method** (FTP server credentials or email) and format. If the vendor uses X12 850 or XML, SSCS will format the order accordingly (this usually is handled by an add-on or by their support team – you may need a spec file from the vendor). When you “**send order**” from CDB, it will either place a file on the FTP or send an email automatically to the vendor, depending on configuration. The key is to verify that the order content (item identifiers and quantities) align with what the vendor expects. Testing a small order is recommended. Once an order is successfully transmitted, the vendor will prepare the shipment and typically generate an invoice (or ASN).

Receiving ASNs / Pre-Delivery Notices: Some vendors will send an **Advanced Shipping Notice** prior to or along with the actual delivery. In the convenience store context, ASNs might be called **Pre-Delivery Notices (PDN)**, especially when using X12 standard 894 ⁴² ⁴³. SSCS CDB does support importing these notifications for certain vendors. According to SSCS's vendor integration list, a couple of distributors (e.g. Cone Distributing, Kay Beer) send **X12 894 (PDN)** documents via AS2, which are verified in the system ¹⁰. An ASN/PDN typically lists the products and quantities that are en route to your store. The process to handle them in CDB is similar to invoices: the ASN file is placed in the EDI folder or received via AS2, and you run the import (Receive and Convert). The system will likely stage the PDN information – possibly creating a *pending delivery record*. In practice, some retailers might use the ASN as a way to **speed up check-in**: when the truck arrives, you already have the electronic list of expected items, so you can quickly verify against actual and then convert that ASN into a received invoice. SSCS's system, being geared for Direct Store Delivery, can use these electronic records to catch discrepancies. For instance, you could run a report comparing your **original order vs. what's on the ASN vs. what actually arrives**, to catch shortages or substitutions ⁴⁴ ⁴⁵.

Workflow Integration: If you send orders electronically from CDB and the vendor sends back ASNs and then invoices, you have a full EDI loop: 1. **Order (850 or XML Order)** sent to vendor from CDB. 2. **ASN (856 or 894)** sent from vendor to you (CDB) before delivery. 3. **Invoice (810 or NAXML)** sent from vendor to CDB after or upon delivery.

To implement this, ensure **scheduling** and responsibilities are clear: for example, you might send orders on a set schedule (some CDB systems can automate generating weekly orders based on history ⁴⁶). For the ASN, configure the system to accept it – SSCS might require the ASN files to have a certain naming or format. When the ASN is imported, verify that it created an “open invoice” or pending receiving document in CDB. Then, when the actual invoice arrives (often the ASN and invoice might be the same in DSD scenarios, or the invoice could just confirm the ASN quantities), import the invoice to finalize the financials.

Tips: Not all vendors will provide ASNs. Many will go straight to sending the invoice (especially if the delivery is brought by a salesperson or driver who finalizes the invoice on site). If you *do* get ASNs, use them to prepare for receiving: match the ASN to the order and note any differences (SSCS can alert you if the ASN shows items you didn't order, etc., similar to how it alerts for vendor substitutions) ⁴⁴. Also, test the ASN import with the vendor – this is less common than invoice import, so sometimes mapping issues might appear. For example, an ASN might use a UPC for items whereas the invoice might use vendor item codes; make sure both will match your item records.

In summary, **SSCS supports a full EDI lifecycle**: sending electronic orders out and receiving delivery notices and invoices in. Implementing this means configuring CAO for that vendor (with correct transport and file format) and enabling ASN/PDN import. The payoff is a highly automated reordering process: the system generates an order based on real sales data, the vendor fulfills and sends a digital notice/invoice, and you receive the product into inventory with a few clicks – all data matched up ⁴⁰ ³⁷.

5. Accepted File Formats and Examples (NAXML, X12, CSV, etc.)

SSCS CDB is flexible in the file formats it can accept for EDI, but the format must conform to one of the known standards or vendor-specific specs. Here we outline the common formats:

- **NAXML (XML):** As discussed, NAXML is a primary format for invoices and item data. It's XML-based, human-readable, and designed by Conexus (formerly PCATS) for the c-store industry ²¹. SSCS's NAXML invoice format (see Section 3 for structure) is essentially an XML **ItemSynch/ItemPrice document** ⁴⁷ ²⁴. Example excerpt of a NAXML invoice file:

```
<Invoice>
  <InvoiceNumber>233817</InvoiceNumber>
  <InvoiceDate>2025-08-25</InvoiceDate>
  ...
  <InvoiceDetail>
    <LineItem>
      <InvoiceUnit>
        <InvoiceUnitId identType="GTIN">00615260026002</InvoiceUnitId>
        <InvoiceUnitDescription>SUGAR HOUSE VODKA 1750ml</
InvoiceUnitDescription>
        <InvoiceUnitQty cstoreUOMBasis="each">1</InvoiceUnitQty>
        <InvoiceUnitCost currency="USD">182.35</InvoiceUnitCost>
        <LineItemGrossAmt>182.35</LineItemGrossAmt>
        <LineItemNetAmt>182.35</LineItemNetAmt>
      </InvoiceUnit>
      <RetailUnitPricing>
        <RetailUnitId identType="GTIN">00615260026002</RetailUnitId>
        <RetailUnitQty>1</RetailUnitQty>
        <RetailPrice currency="USD">227.94</RetailPrice>
      </RetailUnitPricing>
    </LineItem>
    <!-- ...more LineItem... -->
  </InvoiceDetail>
  <InvoiceSummary>
    <SubTotal>2006.42</SubTotal>
    <Tax>0.00</Tax>
    <InvoiceTotal>2006.42</InvoiceTotal>
  </InvoiceSummary>
</Invoice>
```

This snippet (based on a real example) shows how each item line is described with GTIN (UPC), descriptions, quantities, costs, and the summary totals ⁴⁸ ⁴⁹. Note that in actual SSCS usage, this `<Invoice>` section is encapsulated in a broader `<NAXML-BusDoc>` or `<ItemSynch>` document with header info and processing instructions, as shown earlier. **Takeaway:** NAXML XML is the preferred format and covers everything needed (we provided a full template in Section 3).

- **X12 EDI:** Traditional EDI in X12 format (the ANSI ASC X12 standard) is less commonly used directly by SSCS stores, but it is supported via integration partners. The typical transactions would be:
 - **810 – Invoice:** If a vendor sends an 810 (in the 4010 or 5010 X12 version, for instance), you'll likely route it through an EDI network or software (like TrueCommerce, B2BGateway, etc.) which then delivers it to your CDB. SSCS's TrueCommerce profile explicitly lists support for the 810 Invoice for SSCS suppliers ³³ ³². This implies that if your trading partner can only do X12, you can use a service to translate that 810 into the format CDB accepts (most often into the NAXML XML). The content of an 810 includes segments like IT1 for line items, etc., which correspond to the same data (item codes, qty, cost, etc.).
 - **850 – Purchase Order:** If SSCS sends an order, an X12 850 can be generated (though SSCS might generate an XML order and have a translator convert to 850). From the vendor perspective, you may not worry about this if you send via the SSCS-provided method.
 - **856 – ASN:** Some vendors might use X12-856 for ASNs. SSCS's documented PDN support is X12-894 specifically ¹⁰. An X12 894 is a variant used by DSD vendors to announce deliveries ⁴³; it contains similar info to an 856. The structure includes line items with shipped quantities. If your vendor sends 856 or 894, ensure the translator or SSCS system knows how to parse it. The example vendors using 894 (Cone, Kay Beer) show that SSCS can parse those directly when sent via AS2 ¹⁰.
 - **824 – Application Advice:** While not asked in the question, note that some EDI systems send back functional acknowledgements or error reports (824). If your vendor expects an EDI acknowledgment, coordinate with your EDI provider. Generally, when using the edelivery email method, this isn't used – it's more of a one-way import.

Example: An X12 810 file snippet (for context) might look like:

```
ISA*00*                *00*                *ZZ*VENDORIDSEND    *ZZ*YOURIDRECV
*210825*1030*U*00401*000000001*0*T*>~
...
BIG*20250825*233817***DI*20250825~
REF*IA*DABS~
N1*ST*Hills & Hollows Package Agency*92*HILLS_HOLLOWS_BOULDER~
N1*SU*Utah Division of Alcoholic Beverage Control*92*DABS~
IT1*1*6*EA*30.39*PE*UP*00615260026002*VN*039593~
PID*F****SUGAR HOUSE VODKA 1750ml~
... (more IT1/PID for each item) ...
TDS*200642**0~
CTT*10~
SE*??*??~
GE*1*1~
IEA*1*000000001~
```

This is purely illustrative: it shows an invoice 233817 on 8/25/2025, a Ship-To and Supplier, and one line item (IT1) with quantity 6 EA at unit price 30.39, UPC and vendor code included, followed by a PID description. The totals (TDS) here indicate \$2006.42 total. Such an 810 would be translated to the XML structure we showed earlier (6 bottles at \$30.39 cost -> \$182.35 line cost, etc.). **The key point** is that X12 can carry the same info, but you will likely rely on a VAN or EDI translator to interface with SSCS, since SSCS itself uses its internal formats for import ⁵⁰.

- **CSV or Tab-Delimited Files:** A number of regional vendors and third-party inventory companies use simpler delimited text files. For instance, SSCS's "Third-Party Inventory Vendors" list shows many companies providing **tab-delimited or comma-delimited files** with inventory counts ⁵¹ ⁵². Similarly, some vendors might send **CSV invoice files** (with columns for UPC, description, qty, cost, etc.). SSCS can import these if they match an expected layout. Typically, SSCS will provide a spec or you provide the sample to them to configure. An example CSV invoice might have headers like: `ItemCode,Description,PackSize,Quantity,CostExt,TaxExt`. You would place this file in the EDI folder; an SSCS converter can pick it up. In the Central Price Book manual, it's noted that file types are placed in the EDI folder under site-specific subdirectories ⁵³. You may restrict by file mask (e.g. ".csv") if multiple types mix in one folder ³⁴. If using CSV, ensure the encoding is acceptable (UTF-8 or ASCII) and that special characters (commas, quotes) are handled. Example:* Atlantic Dominion might send a CSV invoice – by coordinating with SSCS, you set the import to parse it into the system's fields.
- **PDI Format (Fixed-width):** Many legacy systems (like PDI and older vendor systems) use fixed-length record formats. The "7500 series" files mentioned are PDI's standardized format for item data, and similar structures exist for invoices. These are text files where each line has specific columns for data (e.g., columns 1-7 = item code, 8-37 = description, etc.). SSCS explicitly lists support for PDI 7500 for item updates ¹³. If a vendor uses a fixed-width invoice file, you'll need that layout specification. Once configured, CDB will read each line and create invoice entries. Always test with a short file first because spacing issues can cause misalignment.

File Delivery and Examples: No matter the format, files are usually delivered to the **CDB EDI folder** or via the **EDI email pipeline**. On a typical on-premises setup, SSCS has an `...\SSCS\CDBWIN\EDI\` directory with subfolders for each site (e.g. `001` for site #1). Files that arrive via email get saved there automatically when you do "Receive and Convert," then moved to an archive (often an `EDI\...\BAK` folder) after processing ⁵⁴ ³⁶. You might see filenames like `VENDOR_INV_12345.na.xml` for a NAXML invoice or `INV0001.TXT` for a text invoice. Check with SSCS if a specific naming convention is needed – some older systems might require names like `VNNNN####.TXT` (with vendor number). In modern versions, this is usually not strict, as the contents of the file will identify the vendor (e.g. a vendor ID field in XML, or a coded line in text). For example, the **VendorID in the XML** (like `<VendorID>DABS</VendorID>`) tells CDB which vendor record to tie the invoice to ⁵⁵ ⁵⁶.

Real-World Reference: SSCS's official documentation indicates that for each supported vendor, there is a known file format and location – e.g., "McLane – NAXML ItemSynch/ItemPrice" or "Core-Mark – PDI 7500" ¹⁴. Your goal is to ensure your incoming files match one of those. If a vendor says "We can send CSV or X12," choose one and stick to the needed spec. If you're unsure, involve a **data translation provider**. Companies like TrueCommerce or ChimpKey can take virtually any format the vendor outputs and **convert it in real-time** to your required format ⁵⁷ ⁹. For instance, ChimpKey can turn PDF or Excel invoices into NAXML XML automatically ⁵⁸ (useful if a small vendor only has PDFs – you could forward them to such a service and get back an XML for CDB).

In conclusion, **SSCS accepts NAXML/XML as the primary EDI format**, with support for X12 via integration and text formats as needed. The critical part is to adhere strictly to the expected file layout for your vendor. We've provided examples for clarity: make sure your files include all required fields and are named/transmitted correctly so the system will pick them up ⁷ ⁴⁷.

6. Field-by-Field Mapping of Required EDI Elements

To ensure a successful import, the EDI data fields must map to SSCS's internal fields. Below is a **field-by-field breakdown** of the key elements required in an SSCS EDI invoice file, along with their meanings:

- **Vendor ID** – A code identifying the vendor. This must match the Vendor ID in your CDB system (for example, “DABS” or a number code). In NAXML, this is in `<VendorID>` ⁵⁹. In X12, it could be in an N1 segment or REF segment. **Purpose:** Tells CDB which vendor record to apply this invoice to ⁶⁰.
- **Vendor Name** – The name of the vendor (for human verification). In XML it's `<VendorName>` ⁵⁹. Not strictly used for matching (the ID does that), but SSCS may display it in logs or errors for clarity.
- **Invoice Number** – The invoice identifier from the vendor ⁶¹. This goes into CDB's accounts payable invoice number field. It should be unique for that vendor (CDB will typically warn if a duplicate invoice number is imported to prevent double-entry). In NAXML: `<InvoiceNumber>`; X12: BIG02 segment. Make sure any prefixes the vendor uses are included (e.g., “INV-1001”).
- **Invoice Date** – The date of the invoice ⁶¹. Format is typically `YYYY-MM-DD` in XML. CDB uses this as the document date for posting.
- **Ship-to or Store ID** – Identifies which store location the invoice is for. In multi-site setups or central databases, this is crucial. NAXML uses `<StoreLocationID>` or a `<ShipTo>` party with an `ident` attribute ⁶² ⁶³. Ensure this matches your store's code as known to the vendor or as set in SSCS. If you have one store, this might be omitted or constant.
- **Total Items count** – A count of line items on the invoice ⁶⁴ ⁶⁵. CDB uses this for validation (it should equal the number of `<Item>` entries or lines). On import, SSCS checks consistency (ItemCount should match actual lines) ²⁸.
- **Customer Account Number** (optional) – Many vendors assign you an account # in their system. If provided, include it (e.g., `<CustomerNumber>`). This helps if you have multiple accounts with the same vendor. Not required, but good practice ⁶⁶.
- **Line Items Section:** Each **Item** in the invoice should contain:
 - **Item Identifier (PLU/UPC)** – This is the core field for matching the product. SSCS's price book typically uses a PLU (which could be the UPC or an internal ID). In the XML template, we see `<PLU>` and also an optional `<UPC>` field ⁶⁷ ⁶⁸. **Mapping:** Ideally, the file provides the 12-digit UPC in a UPC field. SSCS will try to match by UPC first if configured that way. If UPC is missing, it may try the PLU or VendorItemCode. Ensure at least one of these matches an existing item record. For new items, SSCS will use these to create the record (UPC is preferred). *Note:* If your system's primary key is UPC, use UPC in the file; if you use an internal SKU or PLU, that should appear (maybe in PLU tag). Vendor's item code should also be included (see below).
 - **Item Description** – The name of the product. Tag: `<ItemName>` ⁶⁹. This is used when creating new items or for human reference. It should closely match the product's actual description as you want it in your price book (you can always edit later, but a good description helps).
 - **Quantity** – The quantity delivered *of the selling unit*. This one can be tricky: in the NAXML ItemSynch format, there may not be an explicit `<Quantity>` tag (some implementations omit it) ⁷⁰ ⁷¹. Instead, quantity might be **inferred** by dividing extended totals by known unit price (see Section 7 for details on inference). However, if the format allows, it's ideal to include a quantity field. For example, in a true “Invoice” XML (not the simplified ItemSynch), there is `<InvoiceUnitQty>` which was 1 in our example (meaning 1 case or unit) ⁷² ⁷³. If your vendor file format has a quantity column (like a CSV “Quantity” or X12 IT1 with quantity), make sure it reflects the number of selling units delivered. If they supply “cases” and you track “eaches,” clarify which one is in the data. *Recommendation:* Include `<Quantity>` in the XML if you have control, even if it's technically

redundant – it makes the file clearer ⁷⁰ . But if not, ensure Price/Cost reflect the total for whatever quantity was delivered (e.g., if 5 cases delivered, the cost field should be 5 * case_cost).

- **Cost (Extended Cost)** – The total wholesale cost for that line item (not per-unit, but for the quantity delivered) ⁷⁴ . In NAXML, `<Cost>` under each Item is expected to be extended cost ⁷⁴ . For example, if 10 units delivered at \$2.00 cost each, Cost = 20.00. SSCS will use this to post to accounts payable (and to calculate unit cost if needed by dividing by qty). Make sure the cost in the file is what *you pay for that line*, excluding any sales tax (tax is separate). If the vendor provided any discounts or allowances on the invoice, those typically should already be factored into the LineItemNetAmt or similar – SSCS expects the net cost. (Any invoice-level discounts could also be in the totals or as separate lines.)
- **Price (Extended Retail)** – The total **retail value** of that line, i.e. the sum of the retail price of each unit * times quantity ⁷⁴ ²⁸ . This field might be less familiar, but SSCS uses it to track margins and to update your price book if the vendor is suggesting a new retail. In the NAXML schema, `<Price>` in each Item is the extended retail amount ⁷⁴ . For instance, if those 10 units have a shelf price of \$3.00 each, Price = 30.00 for the line. Having this field allows SSCS to compute margin on the invoice and flag any changes in retail pricing. Ensure this is provided if the vendor includes it. If the vendor file doesn't contain retail prices, you can leave it blank or set it equal to cost (though that's not ideal). Some vendors do provide an MSRP or "extended retail" – include it. SSCS will compare it to your current sell price and can alert you if the MSRP differs, so you can adjust shelf price ¹⁹ . In our earlier example, for 6 bottles of vodka with \$37.99 each retail, extended retail was \$227.94, which was provided in the file ⁷⁵ .
- **Unit of Measure / Pack Size** – The format should indicate the packaging of the item. In our XML, `<Size>` or sometimes a part of description was used (like "1750ml" or "6x1L") ⁷⁶ ⁷⁷ . This helps identify the item uniquely (especially if the same item comes in different pack). It might not directly import into a field (CDB has a "Size" or "Units in Case" field in inventory), but it's good to have for human reference and item creation. If the vendor provides a separate "pack" or "quantity per case" number, ensure it's either incorporated into the description or handled by the quantity logic. E.g., vendor might say "1 CASE of 24" – you could have Qty=1 and Size="case(24)" or Qty=24 each, depending on convention.
- **Category/Department** – Vendors sometimes classify items by category (like "WINE" or "SNACKS"). SSCS's invoice format includes `<Category>` for each item ⁷⁸ ⁷⁷ . This is useful if the vendor's category aligns with your departments – it can help automatically assign a new item to the correct department in your system. It's not strictly required for import (if missing, the item might default to a miscellaneous category or require manual categorization). But if provided, SSCS will record it. Use the standard category names that your system expects (often the vendor's category might be close; you can map or adjust later). Including it is a best practice.
- **Vendor Item Code** – The code that the vendor uses internally for the item (if different from the UPC or PLU). In the XML template, `<VendorItemCode>` is included for each item ⁷⁶ ⁷⁷ . This might be identical to PLU or totally different. It's important for cross-reference, especially if later you need to discuss the item with the vendor or order it. SSCS stores the vendor's item code in the item's record (there's a field for "Vendor item #"). So providing it means that even if a UPC is missing or changed, you have that linkage. Always include VendorItemCode if available, even if it seems redundant.
- **LastUpdated** – A timestamp field for the item entry. In the format, `<LastUpdated>` is the date-time the data was last changed ⁷⁹ ⁸⁰ . This might be when the invoice was generated. It's used mostly for informational purposes or if using Central Price Book (to see when an item price was last updated). Ensure it's in ISO format (e.g., `2025-08-25T15:12:38Z`). If not provided, SSCS might fill it with the import time. It's optional but recommended.

- **Status** – Typically “Active” for items that are currently sold ⁷⁹. If a vendor were to send an item file of new/discontinued items, this might matter more. In an invoice context, you’ll nearly always use “Active” (since discontinued items wouldn’t be shipped). SSCS will ignore unknown status values, so stick with “Active” unless instructed otherwise.
- **UPC / EAN** – The 12- or 13-digit barcode number. As mentioned under Item ID, this is critical for matching. In the XML, `<UPC>` holds this ⁸¹ ⁸². Provide the full UPC with leading zeros if any, no dashes/spaces. If an item doesn’t have a UPC (e.g., some fresh product or misc), leave it blank but include a `<UPCNote>` explaining (the system won’t reject if UPC is blank, as long as some other code identifies the item). If you have a **check-digit issue** (some vendors drop the check digit in their data), you can supply both versions: for example SSCS uses a `<VerifoneUPC>` (11-digit) in case your POS requires it ⁸¹ ⁸². This is optional; it’s mostly relevant if you have a Verifone POS that uses 11-digit UPCs. The `UPCVerified` flag should be “true” if you are confident the UPC is correct, “false” if not ⁸¹ ⁸². This flag is informational; if false, SSCS might prompt you to confirm the UPC when reviewing new items.
- **Invoice Totals:** After all items, the file should have a summary:
 - **SubTotal** – Sum of all line item extended costs (the total merchandise cost before tax) ⁸³. SSCS will compare this to the computed sum of the Cost fields of items as a validation ²⁸. It populates the invoice subtotal in Accounts Payable.
 - **Tax** – Total tax amount on the invoice ⁸³. If your vendor charges sales tax on some items (often, most C-store goods are tax-exempt for resale, except certain beverages or other items depending on jurisdiction), this would be the sum of tax. Usually this is 0.00 on wholesale invoices, but if not, include it. The system will use it to post tax in accounts payable (and possibly to update item tax flags).
 - **Total** – The invoice grand total (SubTotal + Tax, plus any misc fees) ⁸³. This is the amount that will be booked as the accounts payable liability. SSCS uses it to ensure nothing got lost in import. It should match the sum of what’s on the paper invoice.
 - **ItemCount** – Number of line items (should match the count of `<Item>` entries and the header `TotalItems`) ⁶⁵.
 - **RetailTotal** – Sum of extended retail values of all items ²⁸. This is essentially the total sales value of the delivered goods at your retail prices. SSCS can use this to calculate overall margin for the delivery (RetailTotal vs SubTotal gives an idea of markup). It’s also used in cases where pricebook updates happen – e.g., if the vendor suggests new retails and you accept them, this helps verify all retails were included. Always ensure RetailTotal = sum of Price for all items; SSCS may check this ²⁸.
- **Processing Instructions:** The file often ends with flags that tell SSCS how to handle the import ⁸⁴ ⁸⁵:
- **ImportType** – should be set to the correct context. For invoices with item data, SSCS uses “ItemPrice” (meaning it’s an Item & Price update, often how it labels NAXML invoices) ⁸⁶. Other possible types could be “Invoice” or “Order” in other contexts, but use what SSCS expects (their examples use ItemPrice).
- **UpdateExisting** – true/false flag to update existing item records with any new info from this file ⁸⁷. Generally **True**, so that if costs or list prices differ, the item record updates. If you set False, the

invoice will still come in, but it won't alter your price book info (you'd be treating it as a one-time cost perhaps). Best practice: True, because you want your cost to update to the latest.

- **CreateNew** – true/false whether to auto-create new items that are not found ⁸⁸. As discussed earlier, you might set this to False initially (so that unknown items are skipped or flagged for manual entry). However, if you trust the data and want full automation, set it True: then any new UPC/vendor code not in your system will result in a new item record being created on the fly ⁸⁸. That record will have all the info from the invoice (description, category, etc.) filled in. You may still need to manually assign things like department or pricing zone after, but at least it's in the system. If False, make sure to catch those missing items via the logs or “unmatched item” report.
- **NotifyOnCompletion** – whether to notify when done ⁸⁹. True is often used – the system might pop up a message or send an email (depending on configuration) when an invoice is successfully processed. It's up to you; it doesn't affect data, just notifications.
- **UPCCoverage** – a percentage (like 100.0 or 80.0) indicating what portion of line items had a valid UPC ⁹⁰. This is a helpful metric that some SSCS reports show to gauge data quality. If a lot of items came without UPC, you'd see a lower percentage and know to follow up. You can calculate it (count of items with UPC / total items * 100) and include it ⁹⁰. If not, SSCS might calculate internally anyway. It's optional but good to include.

Mapping these fields correctly is crucial. Essentially, each column in a CSV or each tag in XML must correspond to the correct field in SSCS's database. **Common mistakes to avoid:** missing leading zeros on UPCs, not matching the exact Vendor ID, misplacing decimal points (SSCS expects costs/prices with two decimals and no currency symbols ⁹¹), and forgetting to include one of the total fields. By ensuring all the above fields are present and correctly populated, you greatly increase the chances of a **clean import with no errors** ⁹² ⁹³.

For reference, SSCS's own template and support documentation echo these requirements – for instance, they emphasize including UPCs for matching, using extended price/cost for accuracy, and aligning with the official spec ⁶⁷ ⁷¹. Always cross-check your first few EDI files against this list of fields. You can also look at the resulting invoice in CDB after import and ensure all line items and amounts line up with the vendor's original invoice (the CDB invoice report should show the quantities, items, unit costs, and totals). If something is off (e.g., an item didn't import or the total is different), it often traces back to a mapping issue in these fields.

7. Best Practices for Data Validation and Import Success

Implementing EDI means less manual data entry, but it introduces a dependency on data quality. To maximize success, follow these best practices for validating data and ensuring smooth imports:

Complete and Clean Data: Make sure each EDI file is **complete** – all required fields present (as outlined in Section 6) and formatted correctly. Common data issues that cause imports to fail include: missing tags or columns, non-numeric characters in numeric fields, mismatched totals, or unknown vendor codes. To validate, use XML validation tools for NAXML files (if available, validate against the Conexus XSD or DTD – this catches missing closing tags, wrong data types, etc.). For CSV, open in Excel to see if columns align. SSCS will likely reject files that aren't well-formed (e.g., badly structured XML) ⁹⁴.

Quantity Inference and Pack Handling: One nuanced area is **quantity inference**. As mentioned, some formats (like the ItemSynch XML) don't explicitly state the quantity of each item – instead, they rely on

extended cost or price to imply quantity ⁷¹. SSCS can infer the quantity received by dividing the extended cost by the item's cost in the price book ⁷⁵. For example, if the invoice line says Cost \$182.35 and your price book knows the cost per bottle is \$30.39, the system infers 6 units were delivered ⁷⁵. This requires that your price book had the correct unit cost beforehand. **Best practice:** If your format doesn't carry quantity, ensure your price book data (cost per unit, pack size) is up to date for that vendor's items *before* importing. If not, inference might go wrong (you'd see weird quantities like 5.99 units if costs don't match). Alternatively, advocate for including quantity in the EDI file, which removes ambiguity ⁷⁰ ⁷¹.

For pack sizes, decide on a convention: some retailers prefer EDI to always be in the smallest unit (eaches), others use cases. SSCS can handle either, but you must be consistent. If a vendor sends "1 case of 24", and you want that to translate to 24 units in inventory, you have two choices: **(a)** The EDI file shows quantity 24 (each) and perhaps a size "Case of 24" for info; or **(b)** The EDI file shows quantity 1 and size "Case/24", and your item in CDB is defined such that 1 case increases inventory by 24 (i.e., the item's ordering multiplier). Approach (a) is simpler and avoids inference – you just treat everything as eaches in the data. Approach (b) relies on the system's pack settings. **Best practice:** If possible, have the EDI file reflect the *each quantity*. Many wholesalers do this by default (their "quantity" field might already be eaches even if they deliver in cases). If not, ensure the "case-to-each" conversion is correctly set in CDB (unit of measure). To validate, after an import, pick an item that was delivered by case and check that the on-hand increased by the right number. If it only went up by 1 when you expected 24, you need to adjust how quantity is interpreted or the item's case size.

Rounding and Precision: Financial rounding issues can occur. For instance, if a unit cost * quantity yields a fraction of a cent on the extended cost, the vendor might round differently than CDB. SSCS expects cost and price with two decimal places ⁹¹. It's wise to have the vendor send extended totals as the source of truth (which they typically do). That way, any slight rounding difference is already resolved on their side. But be cautious: if you generate your own EDI file (say, you're converting from a PDF), make sure to calculate extended cost by *rounding at the line item level*, not just summing unrounded numbers, to match the invoice. For example, if an item is \$0.667 each and 3 are delivered, the invoice might show \$2.00 (rounded) vs $3 * 0.667 = \$2.001$. Such tiny differences can cause a total mismatch. Some systems have a tolerance, but others will flag an error if totals don't match exactly. **Best practice:** match the vendor's rounding. If the invoice PDF shows \$2.00, use \$2.00 even if the third decimal would suggest \$2.001. SSCS will accept the file as long as the totals align exactly to the penny.

Data Validation Reports: Use SSCS's built-in reports and screens to validate imports. After importing an invoice, check the **EDI or Invoice import log** (if available). SSCS might output an "EDI Conversion Report" that lists each attachment processed and any errors (for example, "Item not found: XYZ123" or "UPC missing for item line 5"). Any such errors should be addressed: - If "item not found," decide whether to create the item or if it was a substitution you don't carry. - If "invalid UPC" or checksum error, correct the UPC and possibly resend the file or manually fix the item in inventory. - If "total mismatch," do a quick audit of line items vs totals to find which line might be off.

Additionally, the **Outside Updates window** (for price book changes) in CPB will list new items or cost changes that came from EDI ²⁰ ¹⁸. This is a form of validation – review that list to ensure all new items make sense (description, dept, tax flags, etc.). The system may color-code or mark items that have issues (e.g., missing UPC or unassigned category) which you can then fix.

Handling Exceptions – Unknown Items & UPCs: As a best practice, in early stages set `CreateNew=false` (so unknown items don't auto-create) ⁸⁸, and instead let them fail import for you to handle. When you import and the log says X items not found, you can manually add those items to the price book (using info from the invoice) and then re-import or simply add the quantities from that invoice manually. Once you gain confidence, you might enable auto-creation. If you do, then implement a **review process**: after each EDI import, filter your item list for items created “today” or in last week to see any new additions and complete their details (assign them to the correct Department, set a selling price if needed, etc.). **Quantity inference pitfalls:** if an item was new and not in the system, and you had auto-create true, the system might not know the pack size or unit cost beforehand, so it could assume the quantity = 1 with cost equal to extended cost. That means inventory would only go up by 1 even if multiple were delivered. The safe approach is to review new items and adjust their pack/price as needed, then adjust stock if required.

Testing in a Safe Environment: If possible, test EDI imports in a **non-production environment** first. You might have a training mode or a test company dataset. If not, consider doing the first EDI import for a vendor when you don't have to immediately rely on the data (e.g., import at the end of day when you can verify and correct overnight). Confirm the inventory updated correctly and the accounts payable entry matches the paper invoice. By catching errors early, you prevent downstream issues (like wrong inventory counts or wrong costs affecting margins).

Backup and Rollback: Understand how to correct mistakes. If an EDI invoice imported incorrectly (say, every item's quantity was doubled due to a mistake), you should know the procedure to reverse it. This could be voiding the invoice in Accounts Payable (which would remove the inventory added) and then re-importing after fixing the data. During setup, perhaps do a trial import of a test invoice, then practice voiding it, to ensure you're comfortable with the process.

Automated vs Manual Validation: You can configure some automated checks. For instance, SSCS's involvement with Connexus includes efforts like **discrepancy alerts** – e.g., automatically flagging when an invoice has an item that wasn't on the order or if an item's cost deviates from last cost by a large percentage ⁹⁵. Utilize these if available: set tolerance levels for cost changes so the system alerts you if, say, an item's cost increased 10% since last order (maybe a sign of error or something to pay attention to). Also, SSCS can highlight **price changes** on an invoice ¹⁹ – ensure that feature is on, so you don't miss the fact that a candy bar now costs more, requiring you to raise its retail.

Rounding & Totals Validation: Always compare the invoice totals in CDB after import to the vendor's invoice document. They should match to the cent. If not, investigate differences in tax calculation or item rounding. If your state has bottle deposits or CRV (container deposit), note that sometimes those are listed separately on paper invoices. Make sure the EDI file accounts for them – either as separate line items or included in costs – otherwise totals will differ. (SSCS does have modules for bottle deposits and such, which might integrate with EDI if formatted properly.)

Keep Audit Trails: SSCS stores imported EDI files (often moving them to an archive folder `EDI\BAK` with maybe a timestamp) ⁵⁴ ³⁶. It's good practice not to delete these immediately. They serve as audit trail – if six months later you find a discrepancy, you can go back to the original EDI file to see what was sent. Also, if needed, you can re-import from those backups if something catastrophic happened (though be careful to not double-post). Some users print an **Invoice Receipt Report** from CDB after every EDI import, which shows what was recorded (for signature or just records).

In summary, treat the EDI import process as you would a sensitive accounting process: **trust but verify**. The goal is to reach a point where 99% of invoices sail through without issues, but early on, a vigilant validation process will help you catch issues (like incorrect mappings or missing data) before they affect your inventory or books. By paying attention to quantity conversions, rounding, and new item handling, you'll maintain data integrity and fully reap EDI's benefits (speed and accuracy) ³⁷ ⁴⁰ .

8. Integration Methods Supported by SSCS (REST API, Direct DB, File Drop)

Traditional File-Based Integration: Historically, SSCS CDB's EDI integration is file-based – meaning it relies on dropping files (in known formats) into specific locations (local folders or email inboxes) for import ⁷ ³¹ . This method is very robust and still the primary way to get data in/out. The **file drop** can be local (if vendors provide files on a USB or through a local network) or automated (via FTP downloads, etc.). As discussed, the “Receive and Convert” function handles fetching emails or AS2 transmissions and converting them into those files on disk for processing ³⁵ ⁹⁶ . For most EDI vendor setups, you will be using this method. It doesn't require custom coding or APIs – just ensuring the files arrive in the right place.

Direct Database (DB) Access: Some advanced users consider writing directly to the SSCS database (e.g., inserting invoice records directly). **Caution:** This is generally *not* supported or recommended unless you're working under guidance from SSCS. The CDB database is proprietary (or possibly uses a database engine like Pervasive/Actian or SQL depending on version), and writing to it directly can bypass business logic. Instead, SSCS provides the EDI import mechanism as the safe conduit. If you wanted to automate things without using the UI, one approach is scripting the movement of files into the EDI folder and perhaps invoking the import on a schedule (if the software supports command-line triggers or scheduled tasks). But actual direct DB integration (like writing an invoice table) is not standard. That said, some integration partners (like for multi-store chains) might pull data *from* the DB for reporting or push data *to* it (like for inventory sync). If you have a centralized system and an IT team, you could technically populate the vendor's data into the tables that CDB uses for EDI staging. However, doing so would require **extensive knowledge of SSCS's schema** and is outside normal support. **In short:** Direct DB integration is possible in theory but not an out-of-the-box feature – rely on file imports or APIs where available instead.

REST API / Web Services: As of 2025, SSCS has been modernizing some aspects of its platform. The integration with Lula Commerce (for real-time inventory and orders) suggests that newer versions of CDBWin have some API endpoints or at least a mechanism to sync data via cloud ⁹⁷ ⁹⁸ . However, those are likely targeted integrations (e.g., a specific plugin for Lula). **General REST API for EDI:** SSCS does not publicly document a generic REST API for uploading EDI invoices. Most likely, they rely on the established methods (email, FTP, etc.) for EDI. The mention of API in the question might be to explore if you can integrate without files. If you desire a more direct integration (say your vendor has a web service to pull invoices), you could develop a script that calls that service and then writes the results to an XML file for CDB. But currently, you wouldn't POST that data directly into CDB via an API call, because SSCS hasn't advertised such a feature.

One related note: SSCS's **Sunray Cloud Hosting** and other cloud initiatives might allow centralized EDI handling. If your CDB is hosted or you have SSCS's cloud, they might provide an interface to drop files via a web dashboard. But under the hood, it likely still lands in the EDI folder of your instance.

Authentication & Security: When using **AS2 or SFTP** for integration, you will have credentials and certificates to manage. For AS2, SSCS or your store likely has an AS2 endpoint set up with the VAN (Value Added Network) or directly with the vendor. This involves an AS2 ID, URL, and certificate exchange. TrueCommerce or similar providers often handle this (they set up an AS2 pipeline to SSCS) ⁹⁹ ¹⁰⁰ . Ensure all security certs are current and tested. For SFTP/FTP, manage usernames/passwords and perhaps IP allow-lists so the vendor can drop files on your server or vice versa. If using the edelivery.com email, authentication is simpler – the vendor just emails to that address. Behind the scenes, presumably SSCS's system (or a service they use) is polling that mailbox. You likely provided a password for that edelivery email when setting up (or SSCS gave you a mailbox and handles it on their side). Keep that secure and don't post it publicly, since anyone who emails that address with a properly formatted file could inject data (the system wouldn't know if it was a valid source or not, it just sees an email).

Scheduling: Decide how frequently to retrieve and send EDI. Some large retailers do it in near real-time (e.g., as soon as a delivery is received, the driver triggers an invoice that gets sent immediately). Others batch overnight. SSCS CDB's "Receive and Convert" can be run on demand (you click it when you expect invoices) ¹⁰¹ . It might also be schedulable – check if there's an automated way (some versions allow scheduling a task to auto-poll the EDI mailbox every X hours). If not, set a routine, e.g., "Every morning, import prior day's invoices" or "Import invoices right after each delivery if electronic." For **orders (CAO)**, you might schedule them after inventory counts or on certain days. For example, if your supplier cut-off is 3 PM, schedule your automated reorder to run at 2 PM and transmit the EDI order by 2:30 PM.

Test Environments: Since you asked about test environments – inquire with SSCS if they offer a **sandbox mode**. Often, the test environment might just be a separate CDB database (like a training company) where you can import without affecting live data. There isn't an official cloud sandbox that vendors can send to, as far as documentation goes. Instead, coordinate with your vendor to send test files marked clearly as test (perhaps using a test account number or invoice range). You can import them, verify results, then void the test invoices out of the live system if you used live for testing. Some vendors support a "test mode" via EDI (for instance, an AS2 test URL or flag in the file indicating test). If so, use that. But many times, test is simply done with small real-looking invoices that you don't actually post to accounting (or that you reverse). **Important:** Make sure test files don't accidentally get into your real numbers – always double-check vendor IDs and invoice numbers (maybe use obvious numbers like 999999 for tests).

Modern Integrations: The integration with Lula (2025) demonstrates SSCS can now do real-time inventory updates to an external system ⁹⁷ . That implies some form of API or at least a data feed. While that's not directly EDI for vendor, it shows SSCS is evolving. If and when SSCS opens a REST API for general use, it might allow pushing an invoice JSON/XML to an endpoint instead of dropping a file. As of now, though, there's no public evidence of a widely available REST API for vendor EDI. So plan on the tried-and-true methods: - For **inbound EDI (invoices/ASNs)**: use email (edelivery) or an EDI VAN to drop files to your server. - For **outbound EDI (orders)**: use built-in FTP/email sending or generate files and send via your own means.

Finally, document your integration settings: note down the email addresses, FTP sites, schedule, etc., as part of your SOP. This ensures if something fails (like an AS2 certificate expiration), you can quickly identify the issue and work with the vendor/IT to fix it.

9. Error Handling, Logs, and Validation Reports in SSCS CDB

When processing EDI, errors can occur. It's important to know where to find logs and how to interpret them, as well as how to correct any issues. SSCS CDB provides feedback in a few ways:

On-Screen Messages: When you click "Receive and Convert," pay attention to any popup messages. Typically, a dialog will say "X message(s) downloaded" and might list something like "Processed invoice #### from Vendor ABC" or conversely "Error processing invoice from Vendor ABC – see log" ³⁵ ³⁶. If an error occurs during conversion, the software usually notifies you immediately. For example, if the XML file is malformed, it might say "Unable to parse XML" or "Unexpected file format." If only certain lines have issues, the system might still import what it can and warn about the rest.

EDI Log File: SSCS maintains an EDI conversion log (often a text file or accessible via a "Logs" button in the EDI interface). This log will chronicle each attempt. It can usually be found in the CDB program directory or accessible through Support Tools. Look for files named like `EDILog.txt` or similar. In the log, entries will include timestamps and any error codes. For example, an entry might read: "2025-08-25 17:45:23 – Received file DABS_20250825_174701_ItemPrice.xml – Error: Item code 087123 not found (line 2) ¹⁰²." Or "Warning: UPC missing for item 'ARETTE CLASICA BLANCO TEQUILA', assigned temporary code." These clues direct you to what went wrong. If the log indicates an item not found and you expect it to exist, check spelling/format of the code in the file. If it truly is new, that tells you to create the item (or set CreateNew true next time).

Open Invoice List / AP Module: After import, the **Accounts Payable – Open Invoices** screen will list the imported EDI invoices (usually flagged as EDI type). If an invoice didn't appear at all, that suggests it failed to import entirely. If it appears but with issues (like wrong amounts), those issues stem from data but not necessarily an "error" according to the system (the system only flags technical errors, not business ones like a cost discrepancy). It's on you to catch those via review.

Validation Reports: In Central Price Book (for multi-site) there's an "Outside Updates Report" which you can run to see all pending updates from vendor files ¹⁰³ ¹⁰⁴. This report shows things like new items and price changes gleaned from EDI. It's useful for validation: e.g., if the EDI invoice had a new item, it will show on that report, possibly marked for acceptance. You should run this report after imports and review if any entries are unexpected. For instance, if an item shows "New" but you know it's not actually new (maybe a UPC didn't match so it thought it was new), that's a sign of a mapping issue.

Additionally, if SSCS is integrated with Conexus standards, it might support generating an **824 Application Advice** or some form of error feedback to the sender. While not commonly used in CDB, some systems send an error file back to the vendor if things don't match (as hinted in Conexus meetings about discrepancy files) ⁹⁵. Check with SSCS if any such feature is enabled – likely not by default, but if a vendor insists on receiving acceptance or rejection acknowledgments, you'd handle that via your EDI provider (e.g., send a functional acknowledgment 997 or an error report manually).

Common Errors and Handling: - *Unknown Vendor ID:* If the file's VendorID doesn't match any in CDB, the import will fail. The log will say unknown vendor. Fix by correcting the ID in file or adding alias in CDB if needed. - *File not picked up:* If the system says "No new EDI messages" but you know one was sent, check that it was addressed properly (correct edelivery email and site code) and that you clicked receive after it was sent. Also check the EDI folder – if the file landed there but didn't process, perhaps the filename

extension or pattern didn't match. For instance, SSCS might look for *.na.xml or *.xml files. In the log or documentation, see if a specific naming is expected (the log excerpt [46] suggests after conversion it saved an attachment with name including "Invoice...na.xml" ¹⁰⁵). - *Parse errors*: These are issues like malformed XML or unexpected format. The log may show the line/column of XML error. Often this could be due to an illegal character (like an ampersand & in a description not escaped). The fix is to correct the file (e.g., change & to &). SSCS might skip a bad character or stop entirely depending on severity. Running the file through an XML validator can help find such issues. For CSV, a parse error might be triggered by a stray comma in a field messing up columns. - *Item not found / not created*: Already covered – either create the item or enable auto-create. If auto-created, you might see a warning but not an error. You'll then find the new item in your database (possibly with a placeholder name if data was incomplete). - *Mismatched totals*: If SubTotal in file ≠ sum of lines, or Total ≠ SubTotal+Tax, SSCS will likely throw an error and not import (to avoid partial data). The log would highlight the mismatch. You'd need to identify which value is off, correct the file, and re-import. Always ensure the totals in the file are consistent ²⁸ ¹⁰⁶. - *Duplicates*: If the same invoice is imported twice, SSCS should prevent double entry. It may log "Invoice 123456 already exists – skipped." Typically, it uses VendorID+InvoiceNumber to check. If you need to re-import an invoice after fixing data, you might have to first delete/void the bad one in CDB to allow re-import.

Post-Import Checks: After each import (especially in early phase), do a quick check: - Go to Inventory of a known item from the invoice: did its on-hand increase by the expected amount? - Check that item's cost: did it update to the new cost on invoice (if changed)? - Open the AP invoice: are all line items present and do extended amounts match the vendor's invoice? Also verify the GL distribution if you use detailed posting (CDB might break out inventory vs tax automatically). - If anything looks off, address it immediately rather than waiting.

Logs for Orders/ASNs: If you are sending orders electronically, SSCS might have an **Order transmission log** or at least confirmations. Make sure to verify that orders were sent (maybe a "Sent Orders" report exists). For ASNs, they would be imported similar to invoices; if an ASN doesn't import, the error handling is similar (unknown item, etc.). If an ASN comes in but you never get the final invoice, you might choose to treat the ASN as the invoice if it had pricing (some do, some don't). However, generally you'll eventually get the invoice.

Support and Troubleshooting: If you encounter mysterious errors, don't hesitate to reach out to SSCS Support ¹⁰⁷ ¹⁰⁸. They can often interpret log files or have tools to analyze your EDI files. Given that EDI is mission-critical for inventory and accounting, resolving errors quickly is key. It helps to provide support with the problematic file and the log excerpt. Many errors have been seen before, so they might have a quick answer (for example, "Oh, vendor X's CSV always has an extra header line – you need to configure the import to skip the first line.").

Error Prevention: A lot of error handling is prevented by thorough setup: making sure all active items are loaded in your system ahead of time, testing format with multiple scenarios (e.g., test an invoice that has a tax line, test one with a negative line if vendor issues credits, etc.), and updating your system when changes occur. For instance, if a vendor changes their format (maybe they upgraded their EDI software and now send a slightly different XML), keep an eye out for any new errors and get the new spec. Regularly review the SSCS **release notes** or vendor bulletins: sometimes they announce deprecations (like in the vendor list we saw a note "As of 10/10/22, email will not be supported by ABL. Retailers using OneDrive will need to forward to edidelivery email" ¹ – such notices are crucial to avoid disruption).

In summary, use the tools SSCS provides – on-screen feedback, log files, and reports – to catch and fix errors in your EDI process. Establish a routine to review these logs after each import (at least initially). With experience, your EDI imports will become very reliable, and you'll know exactly where to look at the first sign of any hiccup.

10. Version-Specific Notes and Confirming Compatibility

SSCS CDB has gone through various versions, and the EDI capabilities may differ slightly between them. It's important to be aware of your software version and any specific restrictions or features it has:

CDBWin Version: By late 2024, SSCS released a new version of CDBWin that included enhanced integrations (like the Lula API support) ¹⁰⁹. Ensure you know which version you are running (check the "About" in CDB). If you're on a much older version (say early 2010s), the EDI module might have different requirements.

Example differences: - Older versions might only support the PDI or proprietary formats and require a special module for NAXML. Newer versions fully support NAXML natively (as evidenced by SSCS's involvement with NAXML since the early 2000s and being an inaugural member of the standards committee ²²). - Some versions might have limits, e.g., maybe a maximum of 999 line items per invoice or issues with very large files – check the documentation if you expect huge invoices. - The **Central Price Book (CPB)** feature that stages updates is more geared toward multi-site deployments. If you have a single-site system without CPB, new items from EDI might insert directly into your local price book (possibly flagged as inactive until you activate them). CPB version differences (like CPB 4.0 vs CPB 3.x) could mean different screens for outside updates. For instance, the CPB 4.0 manual from April 2022 outlines the vendor import setups and outside update handling ¹² ¹⁵. If you're running that version, follow those guidelines. If an older CPB, steps might vary (but conceptually similar).

NAXML Schema Versions: Conexxus (PCATS) NAXML has versions (3.3, 3.4, 4.0, etc.). Confirm which schema your system expects. Usually, the `version="2.0"` in ItemSynch suggests it's using the EB2B 2.0 schema (which correlates to a certain Conexxus standard release) ¹¹⁰. If a vendor is using a significantly newer schema, there might be extra tags that SSCS doesn't recognize. That usually doesn't break things – unknown tags are ignored – but critical differences (like tag names changed or structure changed) could break parsing. So, a strategy: **obtain a sample file from an already approved vendor** (maybe ask SSCS support for a generic example) and compare it to your vendor's file. If they look structurally the same, you're likely compatible. If not, you might need a mapping or an update. SSCS may offer updates or patches to support new schemas. For example, if Conexxus released a new XML format for invoices, SSCS would likely incorporate it in the next version. If you can't update SSCS, then use a translator to convert the vendor's format to the one your version supports.

Software Patches and EDI fixes: Keep your SSCS software up-to-date with patches, especially those related to EDI. For instance, if there was a known bug in version X where EDI 894 import had an issue, SSCS might fix that in a patch. Reading through release notes or talking to support can reveal if you need any hotfix for certain EDI features.

Testing Version Compatibility: When you first set up, do a thorough test: - Use a **small invoice** file that contains a bit of everything (tax, multiple items, maybe a new item). - Import it and verify each element came through. If something is not handled (e.g., maybe your version doesn't read <UPCNote> and thus you lose that info – not critical, but just to know). - If an element is not recognized, either it's extraneous (which is fine) or it's important (then you may need to adjust the format to fit what the version knows). For

example, maybe older SSCS expected a root tag `<EB2BInvoice>` instead of `<ItemSynch>`. If that's the case, your vendor/translator might need to output the older root tag, or you upgrade.

Multiple EDI Interfaces: Some SSCS users might have both the legacy EDI interface and a newer one concurrently (especially during transition). Make sure you're using the right one. If your version has a new "EDI Dashboard" or similar, utilize that. Confirm where the **EDI folder path** is in your config – version changes might relocate it or require a different folder name.

Backup of Settings: Document any config changes you make for EDI (such as file masks, zone mappings in CPB, etc.). If you upgrade SSCS, re-check these settings afterward – occasionally, an upgrade might reset or change how EDI is configured (for instance, moving from INI file settings to registry). Knowing your settings ensures a smooth transition.

Vendor Version Compatibility: This refers to the vendor's side – ensure the **vendor's EDI version** is compatible with SSCS's. For example, McLane might say "we support NAXML ItemSync 4.0" – verify that SSCS can handle that (likely yes, given they list McLane NAXML support ¹¹¹). Another example: a vendor might initially send 810 X12 version 4010 but later switch to 5010. Your translator should handle that if properly configured. As long as the output to SSCS remains consistent (you abstract that detail out), you're fine.

Strategy if Incompatibility Found: If you do find a version mismatch, you have a few options: - **Update SSCS:** If you're running an old release and a new one supports the needed format, plan an upgrade (keeping in mind to schedule it appropriately). - **Ask Vendor for alternate format:** Many vendors can downgrade their format if needed. E.g., they might be able to send a CSV or an older XML if your system can't do the newest. This might be temporary until you upgrade. - **Use Middleware:** Employ a middleware (like a small script or an EDI service) to convert the format. For example, if the vendor's XML has `<InvoiceDetail>` structure not recognized, a script could transform it into the older format that SSCS knows, then feed it in. This requires some technical effort but can be a lifesaver if you can't immediately change either system.

Version-specific restrictions to note: - Some versions of CDB might not support multi-threaded EDI imports. So import one file at a time; don't drop 50 files in the folder and hit convert, unless support says it can queue them. Usually, it processes sequentially though. - Very old CDB versions might not support AS2/Email automation – meaning you'd manually place files. If you're on such a version and need automation, consider upgrading or using an external script to drop the files then trigger CDB. - Check if your version requires any additional licensing for EDI. SSCS typically bundles EDI in CDB, but ensure your license covers the EDI module (most do if you have Inventory Management). If not, you may need a license key update. - **Conexxus updates:** Conexxus occasionally updates standards (like adding fields for new regulatory requirements). Stay informed via SSCS or industry news if something like that will affect invoices. For instance, if a new tax field was added to NAXML for some reason, older SSCS wouldn't know it – you'd want to strip it out or upgrade.

Confirmation Steps: After everything is live, periodically confirm things are still compatible: - If SSCS gets updated or patched, re-test an EDI import afterward to ensure nothing inadvertently broke. - If a vendor merges or changes EDI providers, revalidate their files (the structure might subtly change). - Annually, do an EDI audit: pick a random EDI invoice and a non-EDI (paper) invoice and ensure they both result in the same data in the system. This helps confirm your EDI process isn't missing anything.

In summary, manage version compatibility by knowing your SSCS version capabilities and your vendor's EDI version. When in doubt, **communication is key**: talk to SSCS support with the exact format you plan to use – “Is this NAXML 3.5 invoice okay for CDB version X?” – they can confirm. Likewise, when vendors refer to standards, align those with what you implement. With proper version control and occasional testing, you'll avoid nasty surprises and have a smooth-running EDI operation.

11. Additional Guidance: Screenshots, Examples, and Resources

(While a text-based format is used here, you should refer to official documentation screenshots or system guides for visual assistance. We'll describe what you would see in the interface and point to resources.)

EDI Setup Screens (Vendor Config): In the CDB software, navigate to **Vendor Maintenance** and then to an EDI or Electronic Invoicing tab (if available). Here you might see fields for “EDI Vendor ID” (which should match the VendorID in files) and options like “Auto-Accept EDI Invoices” or “Use EDI for this vendor.” Ensure those are set appropriately (auto-accept just means invoices flow in without extra approval; you'll likely enable it after initial trust is built). If using Central Price Book, go to **Central Price Book -> Vendor Imports**. There you can “Add” a vendor feed. A form will let you choose the format (a dropdown of supported formats, e.g., “McLane NAXML” or “PDI 7500”) ¹¹² ¹⁴, the site or zone to apply to, and file location mask ³¹ ¹¹³. Fill those in as per your vendor. For instance, select “NAXML ItemSynch/ItemPrice” for McLane ¹¹¹ or “PDI 7500” for Core-Mark, etc., and set file mask or if needed. **Screenshot Tip:** The CPB User Guide PDF (page 92-93) shows an example of this setup screen with fields labeled (Import Type, File Location, File Mask, etc.) ¹¹² ¹¹⁴ – reviewing that image can be very helpful.

Processing EDI Invoices in CDB: After sending your EDI file, open the **EDI Conversion** tool in CDB (under Inventory or a dedicated EDI menu). **Screenshot/Expectation:** The window typically has a “Receive and Convert” button. When you click it, a small dialog appears indicating it's checking for messages. If an invoice is found and processed, you get a message like “1 Invoice converted” ¹¹⁵. If you then go to Accounts Payable -> Invoices -> Open Invoices, you will see the new invoice listed with perhaps a source indicator (some systems mark EDI imports with a special icon or note). You can open it like any invoice to view details. **Verify** the line items, and then you can post it to update financials.

If the invoice didn't come in, check the EDI Conversion log or the popup for errors (for example, a popup might directly say “Error: No matching vendor for VendorID XYZ” or simply “0 messages processed” when you expected one – which implies an issue to troubleshoot).

Official Documentation and Support: SSCS provides user manuals (like the *Computerized Daily Book User's Guide* and *Central Price Book User's Guide*) which contain sections on EDI. These are available on the SSCS portal or provided during training ¹¹⁶. Look up the “Direct Store Deliveries” and “Vendors” chapters for narrative on how EDI fits in ¹¹⁷ ¹¹⁸, and the step-by-step sections for configuration. Also, SSCS's support site might have Knowledge Base articles – e.g., “How to import EDI invoices”, “Troubleshooting EDI import errors”, etc. Don't forget to also utilize industry resources: Connexus has published **guideline documents** for NAXML (available to members or sometimes older versions publicly) ¹¹⁹ ¹²⁰ – these can give deeper insight into field definitions if needed.

Forums and User Groups: If available, join any user forums or groups for SSCS users. Often, other convenience store operators share their experiences with specific vendors. For example, someone might

post “Has anyone integrated with Core-Mark via EDI? I’m seeing some weird values...” and you might find a discussion or a solution. While official docs are the first stop, these community insights are valuable for nuanced cases (like specific regional vendor quirks).

Integrator Documentation: If you’re using a third-party EDI integrator (like TrueCommerce, SPS Commerce, etc.), make sure to obtain their SSCS-specific setup guide. TrueCommerce, for instance, might give you a document saying “For SSCS, we map the 810 like this... and require these fields” ³³. They often handle the heavy lifting, but you should understand what they are doing (in case you need to troubleshoot mappings with them).

Screenshots and Examples: Since we cannot embed actual images here, the best approach is to check the references we cited: - The *SSCS Vendors page* shows a long list of supported vendors and methods ¹ ² – reviewing it gives confidence that your vendor’s format is likely supported (and it hints at what spec to use). - The *Central Price Book User’s Guide (CPB)* contains screenshots of the CPB interface. For example, it illustrates the *Outside Updates* review screen, where you can see items from an imported file waiting for approval ¹²¹ ¹²². It also shows how to accept and distribute those to stores. Even if you’re single-store, the concept of *Outside Updates* might still apply (just directly into your store’s price book). - The *EDI for DABS document* (an internal example we saw) gave a clear look at a well-formed invoice file ¹²³ ¹²⁴ and how the data lines up with real product info. Use that as a template reference. - The “*EDI Delivery Confirmed*” log document showed a real NAXML `<Invoice>` content as processed ⁴⁸ ⁷². This is effectively what SSCS sees and turns into an invoice. Comparing this with the actual invoice data is an excellent training exercise to ensure you understand each part.

Official Support Channels: Keep contact info for SSCS support handy (phone and email) ¹⁰⁷. They can quickly assist if, say, one particular invoice won’t import and you can’t figure out why. Often it might be something small in the file – sending it to them, they may identify the issue (the support team likely has seen many EDI files and errors). SSCS prides itself on support responsiveness ¹²⁵, so don’t struggle alone for too long.

Final Checklist: When everything is configured: - Vendor created in CDB, EDI settings applied (IDs, etc.). - Test EDI invoice file processed successfully (inventory and pricing updated correctly). - Pricing sync configured (vendor item file format set up, if using CPB). - PO ordering tested (if using CAO, ensure test PO reached vendor). - ASN handling tested (if applicable). - Error logs monitored (initial runs show no critical errors; any that appeared were resolved). - Team trained: Your store staff or back-office users should know how to trigger the EDI import and what to do if something is red-flagged (for example, if a new item comes in, who approves adding it? If a cost is way off, who investigates?).

By following this guide and the above checklist, you should be able to onboard a new EDI vendor from scratch and manage the full EDI lifecycle confidently. SSCS’s platform, combined with industry standards like NAXML, will handle the heavy lifting – you just ensure the configuration is correct and maintain oversight with the provided tools. Soon, you’ll find that receiving a truck and processing its invoice is as easy as clicking “Receive” and watching the data flow in accurately ³⁷ ⁴⁰, freeing you to focus on exceptions and analysis rather than data entry.

References:

1. SSCS Direct Store Delivery overview – EDI invoices flow directly into the CDB ¹¹.

2. SSCS Vendors Integration list – supported EDI vendors & file formats (Invoice specs, AS2/email methods) [1](#) [2](#) .
 3. SSCS Central Price Book Guide – configuring vendor file import and accepted format examples [12](#) [14](#) .
 4. Connexus (NAXML) standard – invoice schema includes all key fields (items, quantities, prices, taxes, etc.) for c-store EDI [21](#) [126](#) .
 5. SSCS Internal EDI Template – detailed breakdown of NAXML ItemSynch invoice fields and best practices [25](#) [26](#) .
 6. SSCS EDI Example (DABS) – sample invoice data showing VendorInfo, Item entries (UPC, Cost, Price, etc.), and totals [55](#) [124](#) .
 7. SSCS EDI Log Example – system output of a processed invoice in NAXML format (line items with InvoiceUnit and RetailUnitPricing segments) [48](#) [72](#) .
 8. SSCS Blog – benefits of EDI and automated ordering (accuracy, alerts for price changes, instant inventory updates) [19](#) [37](#) .
 9. Lula & SSCS integration news – confirmation of latest CDB version enhancements (real-time data flow, likely API usage) [109](#) [97](#) .
 10. TrueCommerce on SSCS – notes that standard X12 transaction sets (like 810 invoices) are supported via their EDI network for SSCS [33](#) [32](#) .
-

1 2 5 6 9 10 41 51 52 99 100 111 118 **Vendors – SSCS**

<https://www.sscsinc.com/vendors/>

3 4 55 56 76 78 79 81 123 124 **SSCS EDI for DABS**

<https://docs.google.com/document/d/1AXzdxRw5UaTmUsJYp-0bLtgnvQoVnjbUwHGTUmdBTAc>

7 12 13 14 15 16 17 18 20 31 34 53 103 104 112 113 114 116 121 122 **cpb40.book**

https://portal.sscsinc.com/wp-content/uploads/sites/3/documents/full_manuals/cpb40.pdf

8 24 25 26 27 28 29 30 47 59 60 61 64 65 66 67 68 69 70 71 74 75 77 80 82 83 84 85 86 87

88 89 90 91 92 93 94 106 110 **NAXML Invoice Template for SSCS (Vendor-Agnostic Format)**

<https://docs.google.com/document/d/1dTKRDnRohNIICDhO1Erki8ynPdYM3c1EHMvtOck-Luo>

11 19 44 45 107 108 117 125 **Direct Store Deliveries – SSCS**

<https://www.sscsinc.com/direct-store-deliveries/>

21 23 126 **Retail Merchandise Data Exchange | Conexxus**

<https://www.conexxus.org/retail-merchandise-data-exchange>

22 95 **SSCS Was There: The 2015 Conexxus Annual Conference – The SSCS Blog**

<https://blog.sscsinc.com/2015/05/07/2015-conexxus-conf/>

32 33 50 57 **SSCS - Service Station Computer Systems | TrueCommerce**

<https://www.truecommerce.com/trading-partner/sscs-service-station-computer-systems/>

35 36 48 49 54 62 63 72 73 96 101 102 105 115 **EDI Delivery Confirmed in CDB for SSCS**

<https://docs.google.com/document/d/1qhLDDVhttMNP7hIOtcriaUvq-PMAs-MRSmkGwXRt2TA>

37 38 39 40 46 **EDI, Automated Ordering, and You – The SSCS Blog**

<https://blog.sscsinc.com/2019/10/23/edi-automated-ordering-and-you/>

42 **EDI 894 | Delivery/Return Base Record - Cleo**

<https://www.cleo.com/edi-transactions/edi-894>

43 **EDI 894 Delivery/Return Base Record - TrueCommerce**

<https://www.truecommerce.com/edi-transaction-codes/edi-894/>

58 **Data entry automation, PDF to XML & EDI Conversion - ChimpKey**

<https://chimpkey.com/homepage/>

97 98 109 **Lula Commerce | Lula Commerce Launches Real-Time Integration with SSCS Back-Office System**

<https://www.lulacommerce.com/lulacontent/lula-commerce-launches-integration-with-sscs>

119 120 **Cover Pages: NACS XML Data Interchange (NAXML)**

<https://xml.coverpages.org/naxml.html>